GEOLOGOS ECOSYSTEM: COMPLETE TECHNICAL ARCHITECTURE & IMPLEMENTATION SPEC

Executive Summary

This document provides complete technical architecture for the GEOLOGOS ecosystem:

- **GEOLOGOS-GALAXY GUIDE** (knowledge foundation)
- **Master Toolkit** (tool orchestration)
- **MetaCat** (autonomous LLM environment)
- **LLM Group Chats** (multi-agent workspace)
- **Creative Workflow Hub** (tool bridges)
- **Accessibility Suite** (inclusive access)
- **Mesh Network Layer** (decentralized infrastructure)

Total scope: ~250-300 hours sequential → 3-5 days with 6-8 parallel execution threads.

PART 1: UNIFIED INFRASTRUCTURE ARCHITECTURE

1.1 API Layer (REST + GraphQL)

SERVICE ARCHITECTURE:

```
├── Core API Gateway (FastAPI)
│   ├── Authentication/Authorization (JWT + Role-based)
│   ├── Rate limiting & throttling
│   ├── Request routing & load balancing
│   └── Logging & monitoring
├── Knowledge Service (GEOLOGOS API)
│   ├── Full-text search (Elasticsearch)
```

```
│   ├── Semantic search (embeddings, Milvus/Weaviate)
│   ├── Cross-reference resolution
│   └── Real-time updates (WebSocket)
├── Tool Orchestration Service
│   ├── Tool registry (PostgreSQL)
│   ├── Dependency resolution graph
│   ├── Execution engine (async task queue, Celery)
│   └── Resource management (CPU, RAM, GPU allocation)
├── Agent Service (LLM coordination)
│   ├── Model management (Ollama/LM Studio integration)
│   ├── Prompt template system
│   ├── Context management
│   └── Inter-agent communication
├── Mesh Network Service (decentralized sync)
│   ├── CRDT database (Yjs, Automerge)
│   ├── Node discovery (mDNS, DHT)
│   ├── Packet routing (mesh protocol)
│   └── Multi-transport support
└── Media Service (accessibility, creative)
    ├── Real-time transcription (Whisper)
    ├── Asset management
```

```
    ├── Format conversion
    └── Streaming delivery
```

TECHNOLOGY STACK:

- Backend: FastAPI (Python 3.11+)
- GraphQL: Strawberry or Graphene
- Database: PostgreSQL (relational) + Redis (cache)
- Search: Elasticsearch or Milvus (vector DB)
- Task Queue: Celery + Redis/RabbitMQ
- Async: asyncio, aiohttp
- API Documentation: OpenAPI/Swagger + GraphQL explorer

1.2 Database Schema (PostgreSQL)

```
-- Knowledge/Catalog
CREATE TABLE pillars (
    id SERIAL PRIMARY KEY,
    number INT UNIQUE,
    name VARCHAR(255),
    description TEXT,
    estimated_words INT,
    sections INT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE TABLE sections (
    id SERIAL PRIMARY KEY,
    pillar_id INT REFERENCES pillars(id),
    section_number INT,
    title VARCHAR(255),
    content TEXT,
    keywords TEXT[],
    vector_embedding vector(1536),  -- OpenAI embeddings
    created_at TIMESTAMP
);
CREATE TABLE cross_references (
    id SERIAL PRIMARY KEY,
    source_section_id INT REFERENCES sections(id),
    target_section_id INT REFERENCES sections(id),
    relationship_type VARCHAR(50),  -- "related", "extends", "contradicts"
    strength FLOAT  -- 0.0-1.0 relevance
);
-- Tools Registry
CREATE TABLE tools (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) UNIQUE,
```

```
    description TEXT,
    repository_url VARCHAR(255),
    license VARCHAR(50),
    tags TEXT[],
    dependencies TEXT[],
    install_command TEXT,
    status VARCHAR(20),  -- "ready", "building", "deprecated"
    last_tested TIMESTAMP,
    execution_profile JSONB  -- runtime requirements
);
CREATE TABLE tool_executions (
    id SERIAL PRIMARY KEY,
    tool_id INT REFERENCES tools(id),
    status VARCHAR(20),  -- "pending", "running", "success", "failed"
    input_params JSONB,
    output JSONB,
    error_log TEXT,
    duration_ms INT,
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);
-- Agents/LLM
```

```
CREATE TABLE agents (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    role VARCHAR(50),  -- "researcher", "coordinator", "executor"
    personality_prompt TEXT,
    model_id VARCHAR(100),  -- "llama2", "mistral", etc
    context_window INT,
    temperature FLOAT,
    created_at TIMESTAMP
);
CREATE TABLE conversations (
    id SERIAL PRIMARY KEY,
    agents INT[],  -- array of agent IDs
    title VARCHAR(255),
    messages JSONB,  -- [{role, agent_id, content, timestamp}]
    created_at TIMESTAMP
);
-- Mesh Network
CREATE TABLE nodes (
    id SERIAL PRIMARY KEY,
    node_id VARCHAR(255) UNIQUE,  -- public key or UUID
```

```
    node_name VARCHAR(255),
    transports TEXT[],  -- ["wifi", "lora", "bluetooth"]
    last_seen TIMESTAMP,
    metadata JSONB,  -- capabilities, resources
    is_online BOOLEAN DEFAULT FALSE
);
CREATE TABLE sync_state (
    id SERIAL PRIMARY KEY,
    entity_type VARCHAR(50),  -- "section", "tool", "conversation"
    entity_id INT,
    vector_clock JSONB,  -- for CRDT causality
    last_modified_node VARCHAR(255),
    tombstone BOOLEAN DEFAULT FALSE,  -- for deletion tracking
    updated_at TIMESTAMP
);
-- Accessibility
CREATE TABLE captions (
    id SERIAL PRIMARY KEY,
    session_id VARCHAR(255),
    audio_chunk BYTEA,
    transcript TEXT,
    confidence FLOAT,
```

```
    timestamp_ms INT,
    created_at TIMESTAMP
);
CREATE TABLE preferences (
    user_id VARCHAR(255) PRIMARY KEY,
    accessibility_settings JSONB,  -- {captions: true, high_contrast: true, ...}
    theme VARCHAR(50),
    language VARCHAR(10),
    updated_at TIMESTAMP
);
```

1.3 File Structure (Master Toolkit USB)

```
/master-toolkit/
├── bin/                    # Executables
│   ├── launcher.sh         # Main entry point
│   ├── install-tools.sh    # Auto-install 200+ tools
│   ├── mesh-node.sh        # Start mesh network node
│   └── start-services.sh   # Start all services
├── config/
│   ├── tools-registry.json # 203 tools metadata
│   ├── environment.conf    # System configuration
│   ├── mesh-config.yaml    # Network topology
```

```
│   └── models-manifest.json      # LLM models available
├── data/
│   ├── geologos-knowledge.db     # SQLite backup of GEOLOGOS
│   ├── embeddings/               # Pre-computed embeddings
│   └── cache/                    # Local caching
├── services/
│   ├── api-server/               # FastAPI server
│   ├── mesh-network/             # P2P sync layer
│   ├── llm-coordinator/          # Agent management
│   ├── tool-executor/            # Tool orchestration
│   └── accessibility/            # Captioning, accessibility
├── models/
│   ├── llama2-7b.gguf            # Lightweight LLM
│   ├── mistral-7b.gguf
│   └── embeddings-model/         # For semantic search
├── tools/                        # 203 tool repositories
│   ├── astronomy/
│   ├── chemistry/
│   ├── biology/
│   ├── ... (18 categories)
├── ui/
│   ├── web/                      # React dashboard
```

```
│   ├── cli/                    # Command-line interface
│   └── plugins/                # Plugin system
├── docs/
│   ├── README.md
│   ├── ARCHITECTURE.md
│   ├── GETTING-STARTED.md
│   ├── API-REFERENCE.md
│   └── MESH-NETWORK.md
├── tests/
│   ├── integration/
│   ├── unit/
│   └── performance/
└── scripts/
    ├── build.sh                # Build all services
    ├── test.sh                 # Run test suite
    ├── verify.sh               # Verify integrity
    └── update.sh               # Check for updates
```

PART 2: FRONTEND ARCHITECTURE (UI/UX Layer)

2.1 React Dashboard Architecture

// Main app structure

```
/src/
├── components/
│   ├── KnowledgeSearch/
│   │   ├── SearchBar.tsx
│   │   ├── ResultsGrid.tsx
│   │   ├── SemanticVisualization.tsx
│   │   └── CrossRefGraph.tsx
│   ├── ToolLauncher/
│   │   ├── ToolRegistry.tsx
│   │   ├── DragDropCanvas.tsx
│   │   ├── ExecutionMonitor.tsx
│   │   └── ResourceManagement.tsx
│   ├── AgentWorkspace/
│   │   ├── ChatInterface.tsx
│   │   ├── AgentProfiles.tsx
│   │   ├── ContextPanel.tsx
│   │   └── CollaborationTools.tsx
│   ├── CreativeHub/
│   │   ├── BlenderBridge.tsx
│   │   ├── GimpBridge.tsx
│   │   ├── GodotBridge.tsx
```

```
│   │   ├── AbletonBridge.tsx
│   │   └── AssetManager.tsx
│   ├── AccessibilityPanel/
│   │   ├── CaptioningOverlay.tsx
│   │   ├── ColorContrast.tsx
│   │   ├── AudioIndicators.tsx
│   │   └── PreferencesManager.tsx
│   └── MeshNetwork/
│       ├── NodeVisualization.tsx
│       ├── SyncStatus.tsx
│       ├── NetworkMetrics.tsx
│       └── ConnectivityMap.tsx
├── hooks/
│   ├── useKnowledgeSearch.ts
│   ├── useToolOrchestration.ts
│   ├── useAgentChat.ts
│   ├── useMeshSync.ts
│   └── useAccessibility.ts
├── services/
│   ├── api.ts                # API client
│   ├── websocket.ts          # Real-time connections
│   ├── mesh-client.ts        # Mesh network client
```

```
│   ├── storage.ts              # Local storage management
│   └── accessibility.ts        # A11y utilities
├── state/
│   ├── store.ts                # Redux/Zustand store
│   ├── slices/                 # Knowledge, tools, agents, mesh
│   └── middleware/             # Custom middleware
└── styles/
    ├── index.css
    ├── themes/
    │   ├── light.css
    │   ├── dark.css
    │   └── high-contrast.css
    └── components/
```

2.2 Key UI Components (Pseudocode)

```
// Knowledge Search Component
export const KnowledgeSearch = () => {
  const [query, setQuery] = useState("");
  const [searchType, setSearchType] = useState<"full-text" | "semantic">("semantic");
  const { results, isLoading, error } = useKnowledgeSearch(query, searchType);


  return (
```

```
    <div className="knowledge-search">
      <SearchBar
        value={query}
        onChange={setQuery}
        onTypeChange={setSearchType}
      />

      {searchType === "semantic" && (
        <SemanticVisualization
          results={results}
          onNodeClick={(section) => navigateToSection(section)}
        />
      )}

      <ResultsGrid results={results} />
    </div>
  );
};
// Tool Launcher (Drag-Drop)
export const ToolLauncher = () => {
  const [toolOrder, setToolOrder] = useState<Tool[]>([]);
  const { executeTools } = useToolOrchestration();
```

```
const handleDragEnd = (result) => {
  // Reorder or add tools to pipeline
};

const handleExecute = async () => {
  const execution = await executeTools(toolOrder);
  // Monitor execution in real-time
};

return (
  <div className="tool-launcher">
    <ToolRegistry onAddTool={(tool) => addToExecutionPipeline(tool)} />

    <DragDropCanvas
      tools={toolOrder}
      onDragEnd={handleDragEnd}
      onRemove={(idx) => removeFromPipeline(idx)}
    />

    <ExecutionButton onClick={handleExecute} />
```

```
      <ExecutionMonitor />
    </div>
  );
};
// Multi-Agent Chat Interface
export const AgentWorkspace = () => {
  const { agents, messages, sendMessage } = useAgentChat();
  const { geologosContext } = useKnowledgeSearch(/* current topic */);

  const handleSendMessage = async (content: string) => {
    // Message goes to coordinator agent
    // Coordinator routes to appropriate agents
    // All agents see GEOLOGOS context
    const response = await sendMessage(content, {
      context: geologosContext,
      toolsAvailable: /* available tools */
    });
  };

  return (
    <div className="agent-workspace">
      <AgentProfiles agents={agents} />
```

```
      <ContextPanel context={geologosContext} />

      <ChatInterface

        messages={messages}

        onSendMessage={handleSendMessage}

      />

    </div>

  );

};
```

PART 3: MESH NETWORK ARCHITECTURE (Decentralized Sync)

3.1 Mesh Network Overview

TOPOLOGY:

Online/Offline → Works seamlessly

P2P Sync → No central server required

Multi-Transport → WiFi, LoRa, Bluetooth, Cellular

CRDT Sync → Conflict-free merging

KEY COMPONENTS:

1. Node Discovery (mDNS + Distributed Hash Table)

2. Message Routing (Routing protocol for mesh)

```
  3. Data Sync (CRDT-based eventual consistency)

  4. Conflict Resolution (Automatic via CRDT, human fallback)

  5. Resource Management (Limited bandwidth on LoRa/Bluetooth)

SUPPORTED TOPOLOGIES:

  - Star (central hub with leaf nodes)

  - Mesh (multi-hop routing)

  - Hybrid (online ↔ offline mesh ↔ cloud)
```

3.2 CRDT-Based Sync

```
// Using Automerge or Yjs for CRDT

import * as Automerge from "@automerge/automerge";

interface SyncDocument {

  // GEOLOGOS knowledge (read-mostly)

  sections: Automerge.Map<Section>;



  // Tool executions (write-mostly)

  toolExecutions: Automerge.List<Execution>;



  // Agent messages (append-only log)

  messages: Automerge.List<Message>;



  // Mesh state (node metadata)

  nodes: Automerge.Map<Node>;
```

```
}

// Sync algorithm

class MeshSyncEngine {

  async syncWithPeer(peerId: string) {

    // 1. Exchange vector clocks

    const myState = this.getVectorClock();

    const peerState = await requestVectorClock(peerId);



    // 2. Determine what each side needs

    const myChanges = this.getChangesSince(peerState);

    const peerChanges = await requestChangesSince(myState, peerId);



    // 3. Send/receive changes

    await sendChanges(peerId, myChanges);

    this.applyChanges(peerChanges);



    // 4. Merge with CRDT (automatic conflict resolution)

    // Automerge handles concurrent edits gracefully

  }



  // Selective sync (bandwidth optimization for LoRa/Bluetooth)
```

```
  async selectiveSync(peerId: string, topics: string[]) {

    // Only sync changes related to specified topics

    // Reduces bandwidth on low-capacity links

  }

}
```

3.3 Multi-Transport Support

```
# Transport abstraction layer

class TransportManager:

    def __init__(self):

        self.transports = {

            'wifi': WiFiTransport(),

            'lora': LoRaTransport(),

            'bluetooth': BluetoothTransport(),

            'cellular': CellularTransport(),  # fallback

        }



    def discover_peers(self):

        """Find all reachable peers across all transports"""

        peers = {}

        for name, transport in self.transports.items():

            if transport.is_available():

                peers[name] = transport.discover_peers()
```

```
    return peers


def send_message(self, peer_id, message):
    """Send message via best available transport"""
    # Priority: LoRa (lowest power) → WiFi (fastest) → Bluetooth
    for transport in self.get_priority_order():
        if transport.can_reach(peer_id):
            return transport.send(peer_id, message)
    raise NoRouteError(f"Cannot reach {peer_id}")


def optimize_for_transport(self, transport_name, data):
    """Compress/split data based on transport constraints"""
    if transport_name == 'lora':
        # LoRa: max 250 bytes, split into multiple packets
        return self.chunk_data(data, max_size=200)
    elif transport_name == 'bluetooth':
        # Bluetooth: higher bandwidth, use compression
        return self.compress(data)
    else:
        # WiFi/Cellular: full bandwidth
        return data
```

3.4 Offline-First Architecture

```
// Offline-first sync with eventual consistency

class OfflineFirstManager {

  private localDB: IndexedDB;  // Browser storage

  private remoteDB: API;      // Server/Mesh

  private syncQueue: SyncQueue;



  async write(entity: Entity) {

    // 1. Write to local storage immediately (offline support)

    await this.localDB.put(entity);



    // 2. Queue for remote sync (fire-and-forget)

    this.syncQueue.enqueue({

      type: 'write',

      entity,

      timestamp: Date.now()

    });



    // 3. Try to sync now; if offline, retry on reconnect

    this.attemptSync();

  }
```

```
async attemptSync() {
  if (!this.isOnline()) return;

  while (this.syncQueue.hasItems()) {
    const { type, entity } = this.syncQueue.dequeue();
    try {
      await this.remoteDB.sync(type, entity);
    } catch (error) {
      // Re-enqueue on failure
      this.syncQueue.enqueue({ type, entity });
      break;
    }
  }
}

async read(entityId): Promise<Entity> {
  // Read from local storage (instant, offline)
  // Periodically sync from remote for freshness
  const local = await this.localDB.get(entityId);

  // If online, get remote version (eventually consistent)
```

```
    if (this.isOnline()) {
      const remote = await this.remoteDB.get(entityId);

      // Merge using CRDT rules
      const merged = this.mergeWithCRDT(local, remote);

      // Update local with merged version
      await this.localDB.put(merged);
      return merged;
    }

    return local;  // Return stale data if offline
  }
}
```

PART 4: TOOL ORCHESTRATION ENGINE

4.1 Tool Registry & Execution

```
# Tool orchestration system

class ToolOrchestrator:

    def __init__(self, config_path: str):

        self.registry = ToolRegistry(config_path)
```

```
    self.executor = Executor()
    self.monitor = Monitor()

def create_pipeline(self, tools: List[Tool]) -> Pipeline:
    """Create execution pipeline with dependency resolution"""
    # 1. Validate tool dependencies
    for tool in tools:
        missing = self.registry.validate_dependencies(tool)
        if missing:
            raise DependencyError(f"Missing: {missing}")

    # 2. Resolve execution order (topological sort)
    order = self.resolve_execution_order(tools)

    # 3. Create pipeline stages
    stages = [ExecutionStage(tool, dependencies) for tool in order]

    return Pipeline(stages)

async def execute_pipeline(self, pipeline: Pipeline) -> ExecutionResult:
    """Execute tool pipeline with monitoring & error handling"""
```

```
results = {}


for stage in pipeline.stages:
    try:
        # Allocate resources
        resources = self.allocate_resources(stage.tool)


        # Execute tool
        start_time = time.time()
        result = await self.executor.execute(
            stage.tool,
            input_data=results.get(stage.dependencies),
            resources=resources
        )
        duration = time.time() - start_time


        # Monitor execution
        self.monitor.record(stage.tool, {
            'status': 'success',
            'duration_ms': duration * 1000,
            'output_size': len(str(result))
        })
```

```
    results[stage.tool.name] = result


except ToolExecutionError as e:
    # Log error, optionally retry
    self.monitor.record(stage.tool, {
        'status': 'failed',
        'error': str(e)
    })


    # Decide: retry, skip, or abort
    if stage.tool.on_error == 'retry':
        # Retry logic
        pass
    elif stage.tool.on_error == 'skip':
        # Skip and continue
        continue
    else:
        # Abort entire pipeline
        raise
```

```
        return ExecutionResult(results)


    def allocate_resources(self, tool: Tool) -> Resources:
        """Allocate CPU, RAM, GPU based on tool requirements"""
        available = self.get_available_resources()
        required = tool.execution_profile.resources


        if available.cpu < required.cpu:
            raise InsufficientResourcesError("Insufficient CPU")


        return Resources(
            cpu=min(required.cpu, available.cpu),
            ram=min(required.ram, available.ram),
            gpu=available.gpu if required.gpu else None
        )
```

4.2 Tool Registry JSON Format

```
{
  "tools": [
    {
      "id": "qgis-001",
      "name": "QGIS",
      "category": "geospatial",
```

"description": "Geographic Information System",

"repository": "https://github.com/qgis/QGIS",

"license": "GPL-2.0",

"version": "3.34.0",

"installation": {

  "ubuntu": "sudo apt install qgis",

  "macos": "brew install qgis",

  "windows": "winget install QGIS",

  "docker": "docker pull qgis/qgis:latest"

},

"dependencies": [

  {"name": "GDAL", "version": ">=3.6"},

  {"name": "PROJ", "version": ">=9.0"}

],

"execution_profile": {

  "resources": {"cpu": 2, "ram_gb": 4, "gpu": false},

  "timeout_seconds": 3600,

  "max_parallel": 1,

  "supports_batch": true

},

"inputs": [

```
      {
        "name": "input_layer",
        "type": "file",
        "format": "shp,geojson,geopackage",
        "required": true
      }
    ],
    "outputs": [
      {
        "name": "processed_layer",
        "type": "file",
        "format": "geojson"
      }
    ],
    "tags": ["mapping", "analysis", "data-import", "visualization"]
  }
]
}
```

PART 5: IMPLEMENTATION TASK BREAKDOWN

5.1 Phase 1: Foundation (40 hours) — Parallel Threads

Thread 1: Backend API Services (20 hours)

- ☐ FastAPI server scaffold
- ☐ PostgreSQL schema setup
- ☐ Authentication/JWT implementation
- ☐ Tool registry API endpoints
- ☐ Knowledge search API (full-text)
- ☐ Testing & validation

Thread 2: Frontend Dashboard (15 hours)

- ☐ React project setup
- ☐ Component architecture
- ☐ Knowledge search UI
- ☐ Tool launcher UI skeleton
- ☐ Responsive layout
- ☐ Unit tests

Thread 3: Tool Registry & Automation (5 hours)

- ☐ Generate 203 tool metadata JSON
- ☐ Create tool installation scripts
- ☐ Dependency resolution algorithm
- ☐ Tool validation system

5.2 Phase 2: Integration (60 hours) — Parallel Threads

Thread 1: Knowledge Integration (15 hours)

- ☐ Ingest GEOLOGOS into PostgreSQL
- ☐ Generate vector embeddings (OpenAI API)
- ☐ Semantic search implementation (Milvus/Weaviate)
- ☐ Cross-reference resolution engine
- ☐ API endpoints for knowledge access

Thread 2: Tool Orchestration (20 hours)

- ☐ Tool executor engine (Celery + Docker)
- ☐ Resource allocation system
- ☐ Execution monitoring & logging
- ☐ Error handling & retry logic
- ☐ Tool bridging (UI → tools)

Thread 3: Agent System (15 hours)

- ☐ Agent framework setup (multi-agent design)
- ☐ LLM model integration (Ollama/LM Studio)
- ☐ Prompt template system
- ☐ Agent communication protocol
- ☐ Context management (GEOLOGOS-aware)

Thread 4: Mesh Network (10 hours)

- ☐ CRDT implementation (Automerge/Yjs wrapper)
- ☐ Node discovery (mDNS)
- ☐ P2P sync engine
- ☐ Multi-transport abstraction
- ☐ Offline-first storage (IndexedDB/SQLite)

5.3 Phase 3: Polish & Features (40 hours) — Parallel Threads

Thread 1: UX & Accessibility (18 hours)

- ☐ Creative tool bridges (Blender, GIMP, Godot, Ableton)
- ☐ Real-time captioning (Whisper integration)
- ☐ High-contrast themes
- ☐ Keyboard navigation
- ☐ Screen reader support

Thread 2: Advanced Features (12 hours)

- ☐ Multi-agent group chat UI
- ☐ Workflow visualization
- ☐ Data export/import
- ☐ Plugin system
- ☐ Custom tool creation UI

Thread 3: Testing & Optimization (10 hours)

- [ ] Integration tests (all services)
- [ ] Performance profiling
- [ ] Load testing
- [ ] Security audit
- [ ] Optimization passes

5.4 Phase 4: Deployment (20 hours) — Parallel Threads

Thread 1: Master Toolkit USB (12 hours)

- [ ] Create bootable USB image (Linux base)
- [ ] Bundle all services (Docker containers)
- [ ] Pre-load models & tools
- [ ] Installation scripts
- [ ] Boot sequence & auto-launch

Thread 2: Documentation & CI/CD (8 hours)

- [ ] API documentation (OpenAPI)
- [ ] User guides & tutorials
- [ ] Architecture documentation
- [ ] GitHub Actions workflows
- [ ] Docker Hub publishing

PART 6: CODE SCAFFOLDS (Ready for Implementation)

6.1 FastAPI Main Server

```
# main.py

from fastapi import FastAPI, Depends, HTTPException

from fastapi.cors import CORSMiddleware

from fastapi.responses import JSONResponse

import asyncio
```

```
from datetime import datetime

app = FastAPI(

    title="GEOLOGOS Ecosystem API",

    description="Universal knowledge + tool orchestration",

    version="1.0.0"

)

# CORS configuration

app.add_middleware(

    CORSMiddleware,

    allow_origins=["*"],

    allow_credentials=True,

    allow_methods=["*"],

    allow_headers=["*"],

)

# ============ Knowledge Endpoints ============

@app.get("/api/v1/knowledge/search")

async def search_knowledge(query: str, search_type: str = "semantic", limit: int = 10):

    """Search GEOLOGOS knowledge base"""

    # Implementation: Query Elasticsearch/Milvus

    pass

@app.get("/api/v1/knowledge/pillars")
```

```
async def get_pillars():
    """List all 26 pillars"""
    pass
@app.get("/api/v1/knowledge/pillar/{pillar_id}")
async def get_pillar(pillar_id: int):
    """Get pillar details + sections"""
    pass
@app.get("/api/v1/knowledge/section/{section_id}")
async def get_section(section_id: int):
    """Get section with cross-references"""
    pass
# ============ Tool Endpoints ============
@app.get("/api/v1/tools")
async def list_tools(category: str = None, tag: str = None):
    """List available tools, optionally filtered"""
    pass
@app.post("/api/v1/tools/execute")
async def execute_tool(tool_name: str, input_data: dict):
    """Execute tool with given input"""
    pass
@app.get("/api/v1/tools/execution/{execution_id}")
async def get_execution_status(execution_id: str):
```

```
    """Get real-time execution status"""

    pass

# ============ Agent Endpoints ============

@app.post("/api/v1/agents/chat")

async def agent_chat(messages: List[dict], context: dict = None):

    """Send message to agent system"""

    pass

@app.get("/api/v1/agents")

async def list_agents():

    """List active agents"""

    pass

# ============ Mesh Network Endpoints ============

@app.get("/api/v1/mesh/nodes")

async def get_mesh_nodes():

    """Get all nodes in mesh network"""

    pass

@app.post("/api/v1/mesh/sync")

async def sync_with_peers(selective: bool = False, topics: List[str] = None):

    """Initiate sync with mesh peers"""

    pass

@app.websocket("/ws/knowledge")
```

```
async def websocket_knowledge(websocket):
    """WebSocket for real-time knowledge updates"""
    pass
@app.websocket("/ws/agent-chat")
async def websocket_agent_chat(websocket):
    """WebSocket for multi-agent conversations"""
    pass
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

6.2 React Component Skeleton

```
// src/components/KnowledgeSearch/SearchBar.tsx
import React, { useState, useEffect } from 'react';
import { useKnowledgeSearch } from '../../hooks/useKnowledgeSearch';
interface SearchBarProps {
  onResults: (results: any[]) => void;
  onTypeChange: (type: 'full-text' | 'semantic') => void;
}
export const SearchBar: React.FC<SearchBarProps> = ({ onResults, onTypeChange }) => {
  const [query, setQuery] = useState('');
  const [searchType, setSearchType] = useState<'full-text' | 'semantic'>('semantic');
  const { search, isLoading } = useKnowledgeSearch();
```

```
const handleSearch = async (e: React.FormEvent) => {
  e.preventDefault();
  const results = await search(query, searchType);
  onResults(results);
};

const handleTypeChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  const type = e.target.value as 'full-text' | 'semantic';
  setSearchType(type);
  onTypeChange(type);
};

return (
  <div className="search-bar">
    <form onSubmit={handleSearch}>
      <input
        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search across 730,000+ words..."
```

```
        disabled={isLoading}
      />

      <select value={searchType} onChange={handleTypeChange}>
        <option value="semantic">Semantic Search (AI)</option>
        <option value="full-text">Full-Text Search</option>
      </select>

      <button type="submit" disabled={isLoading}>
        {isLoading ? 'Searching...' : 'Search'}
      </button>
    </form>
  </div>
);
};
```

PART 7: DEPLOYMENT CHECKLIST

- ☐ All services containerized (Docker)
- ☐ Docker Compose for local development
- ☐ Kubernetes manifests for production
- ☐ GitHub Actions CI/CD pipeline
- ☐ Security scanning (OWASP, dependency check)
- ☐ Load testing & performance benchmarks
- ☐ Disaster recovery plan
- ☐ Monitoring & alerting (Prometheus, ELK)

- ☐ USB image creation & testing
- ☐ Documentation complete & tested
- ☐ Community launch plan

PART 8: SUCCESS METRICS

Metric	Target
API response time (p95)	<200ms
Knowledge search latency	<500ms
Tool execution overhead	<2s
Mesh sync latency (WiFi)	<1s
Mesh sync latency (LoRa)	<30s
System uptime	99.9%
GEOLOGOS embedding quality	>0.85 cosine similarity
Accessibility WCAG 2.1 AA	100% compliance

CONCLUSION

This architecture provides: ✅ **Complete knowledge integration** (GEOLOGOS-GALAXY GUIDE) ✅ **Tool orchestration** (200+ tools, automatic execution) ✅ **Multi-agent coordination** (LLM group chats) ✅ **Decentralized infrastructure** (Mesh network, offline-first) ✅ **Accessibility** (Real-time captioning, inclusive design) ✅ **Production readiness** (Containerized, monitored, scalable)

Estimated implementation time: 3-5 days with 6-8 parallel execution threads.

Ready for parallel execution. All documentation complete. Let's build this.